PROJECT SUBMISSION & TECHNICAL ARCHITECTURE REPORT

Conversation AI Copilot for GoHighLevel (GHL)

An Autonomous Multi-Model Action Execution Agent & High-Performance Solutions Architecture for GoHighLevel Sub-Accounts.

Python 3.10+ | FastAPI Core | Google Gemini 3.6 & 3.7 Flash | Groq Cloud LPUs | OpenRouter AI Gateway | GoHighLevel REST API v2 | Server-Sent Events (SSE) | Semantic RAG Engine | Multi-Key Self-Healing Pool

PROJECT DOMAIN
AI Copilot & CRM Automation

IMPLEMENTATION STATUS
Production Ready • Deployed

SUBMISSION DATE
September 2026

Table of Contents

TECHNICAL REPORT

> 📌 **Executive Overview**
>
> This document represents the formal technical submission for the **Conversation AI Copilot for GoHighLevel**. It outlines the end-to-end system design, prompt engineering logic, API specifications, real-time streaming infrastructure, and production vertical CRM blueprints implemented in the repository.

1. Executive Summary & Problem Scope

SECTION 01

1.1 Industry Background & The Friction in HighLevel Setup

GoHighLevel (GHL) is the leading all-in-one CRM and marketing automation infrastructure utilized by over 50,000 marketing agencies and hundreds of thousands of small businesses globally. However, configuring new client sub-accounts requires significant manual effort:

- **Repetitive Taxonomy Setup:** Creating dozens of custom contact fields, standardized tags, and multi-stage opportunity pipelines manually consumes 4 to 8 hours per client onboarding.
- **Fragmented Multi-Channel Messaging:** Sales reps must constantly navigate complex UI menus to trigger outbound SMS, emails, tasks, and internal notes.
- **Provider Rate Limiting & Fragile AI Systems:** Conventional LLM wrappers often hardcode single API keys, resulting in sudden downtime when rate limits (HTTP 429) or token quotas are reached.
- **Hallucinations & Flawed CRM Payloads:** Generic AI chatbots frequently invent non-existent GHL API endpoints or generate malformed JSON payloads that fail silently.

1.2 The Solution: Conversation AI Copilot

The **Conversation AI Copilot** solves these challenges by deploying an autonomous, multi-model execution agent directly integrated with GoHighLevel's REST API v2. Users can interact via natural language prompts to perform complete sub-account audits, deploy vertical CRM architectures in seconds, search/create contacts with strict phone hygiene, and stream interactive execution progress with live tool badges.

⚡ Instant Sub-Second Execution

Leverages Groq Cloud LPUs and Google Gemini Flash to deliver initial token generation under 400ms with native tool calling.

Resilient Key Pool & Failover

Self-healing multi-key pools with automatic 65-second cooldown recovery and dynamic fallback to alternative models on quota depletion.

Turnkey Vertical Blueprints

Pre-packaged schemas for specialized niches (e.g. Gyms & Fitness Centers) provisioning 14 fields, 12 tags, and 2 pipelines in one click.

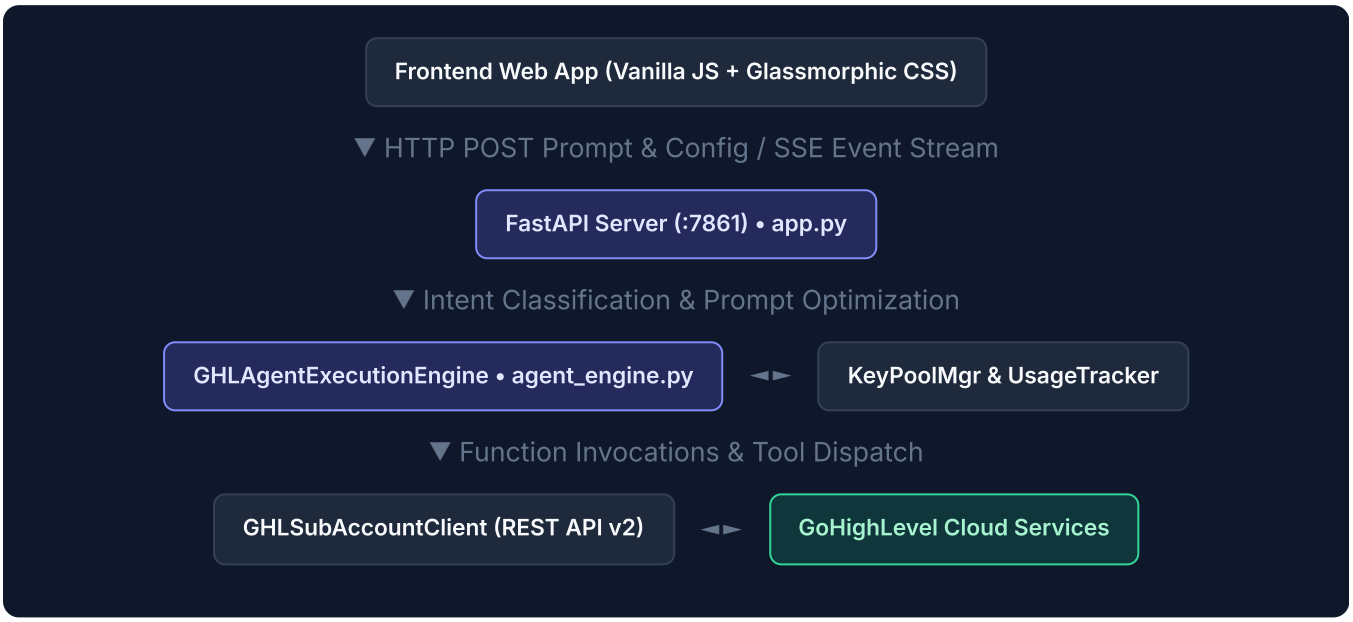
Verified Semantic RAG

Grounded in verified agency documentation (PDF/DOCX) to cite authentic Meta Ads dashboards, KPI reports, and case studies.

2. System Architecture & Topology

SECTION 02

The application is structured into decoupled, high-cohesion architectural tiers ensuring high throughput, seamless streaming, and strict separation of concerns:



2.1 Component Breakdown

COMPONENT	FILE LOCATION	CORE RESPONSIBILITIES
Web Server & Router	`app.py`	FastAPI application, static assets mounting, connection caching (300s TTL), SSE response streaming, and REST API controllers.
Execution Core	`agent_engine.py`	Prompt intent classification, 29 architectural rule injection, Gemini/Groq/OpenRouter dispatching, and function calling loop.
GHL Client SDK	`ghl_client.py`	GoHighLevel REST API v2 wrapper: contacts, pipelines, deals, tags, custom fields, notes, tasks, and conversations.
Key Pool Manager	`key_pool_manager.py`	Multi-key management for Gemini and OpenRouter, credit balance polling, 429 rate limit detection, and 65s auto-recovery cooldown.
Usage Tracker	`usage_tracker.py`	Tracks daily requests, daily tokens, sliding-window TPM/RPM, and writes persistent stats to `model_usage.json`.
Semantic RAG Engine	`portfolio_knowledge_base.py`	Parses PDF and DOCX agency case studies, generates vector embeddings, and performs cosine similarity + keyword overlap matching.
Vertical Blueprint	`gym_architecture.py`	Turnkey schema definitions for Gym & Fitness Center CRM taxonomy, retention pipelines, and lead scoring hysteresis.

3. Multi-Provider AI Engine & Intent Classification

SECTION 03

3.1 Intelligent Intent Classification

The engine does not treat all user inputs identically. Every prompt is evaluated by `classify_prompt_intent()` against compiled regex patterns and keyword sets to select the optimal temperature, token budget, and context window:

INTENT MODE	TARGET INTERACTION	TEMPERATURE	MAX TOKEN BUDGET
`full_build`	Full landing pages, complete CRM architecture, or vertical code bundles.	0.7	Up to 8,192 Tokens
`proposal_or_qa`	Technical audits, client proposals, scope of work, or architectural reviews.	0.2	4,000 Tokens
`iteration`	Styling adjustments, color swaps, tag renaming, or single-field updates.	0.4	2,500 Tokens
`quick_answer`	Concise platform guidance, troubleshooting, or single GHL queries.	0.2	1,500 Tokens
Tool Invocation Mode	Active tool calling (e.g. creating contact or pipeline).	**0.1**	Deterministic Output

3.2 Supported Model Catalog & Capabilities

MODEL IDENTIFIER	PROVIDER	HARDWARE / FRAMEWORK	SPECIALIZATION & RATE LIMIT
`gemini-3.6-flash`	Google AI Studio	Native Google GenAI SDK	**Recommended** • 1M TPM • 15 RPM • Native Tool Calling
`gemini-3.7-flash`	Google AI Studio	Native Google GenAI SDK	Hybrid Reasoning Engine • Complex multi-step CRM automations
`groq/compound-mini`	Groq Cloud	LPU Tensor Architecture	⚡ 70k TPM • Ultra-fast landing page & code generation
`qwen/qwen3.8-27b`	Groq Cloud	LPU Tensor Architecture	Open-weights benchmark leader • High-precision tool calling
`x-ai/grok-4.6`	OpenRouter Hub	xAI Infrastructure	Deep reasoning & real-time conversational synthesis
`anthropic/claude-3.5-sonnet`	OpenRouter Hub	Anthropic API	State-of-the-art coding and SaaS systems architecture

4. Server-Sent Events (SSE) Streaming Protocol

SECTION 04

The backend delivers immediate, token-by-token output and interactive tool status via **Server-Sent Events (SSE)**. This avoids long HTTP blocking pauses while multi-step tool calls execute against external CRM endpoints.

4.1 SSE Event Lifecycle & Schema

EVENT TYPE	JSON PAYLOAD STRUCTURE	CLIENT UI BEHAVIOR
`status`	`{"type": "status", "message": "string"}`	Displays subtle animated status message in chat stream.
`tool_call`	`{"type": "tool_call", "name": "...", "args": {...}}`	Renders an active, pulsing tool execution badge in UI.
`tool_result`	`{"type": "tool_result", "name": "...", "result": {...}, "success": bool}`	Transitions execution badge to green (Success) or red (Error).
`chunk`	`{"type": "chunk", "text": "string"}`	Appends tokenized markdown directly to active response buffer.
`usage_update`	`{"type": "usage_update", "model": "...", "stats": {...}}`	Updates live quota bar and request counts in real-time.
`done`	`{"type": "done"}`	Signals stream termination; enables input controls.

4.2 Real-Time SSE Stream Trace Example

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive

data: {"type": "status", "message": "Evaluating prompt and preparing execution..."}

data: {"type": "tool_call", "name": "create_contact", "args": {"first_name": "Jordan", "last_name": "Be

data: {"type": "tool_result", "name": "create_contact", "result": {"success": true, "message": "Contact

data: {"type": "chunk", "text": "I have created the contact for **Jordan Bell** in your GoHighLevel sub

data: {"type": "usage_update", "model": "gemini-3.6-flash", "stats": {"daily_requests": 18, "daily_toke

data: {"type": "done"}
```

5. Multi-Key Pool & Self-Healing Resilience

SECTION 05

Production AI systems frequently encounter third-party rate limits. The `key_pool_manager.py` module implements automated rotation, credit tracking, and self-healing cooldown logic to ensure uninterrupted uptime.

5.1 Gemini Key Pool (`GeminiKeyPool`)

- **Multi-Key Ingestion:** Ingests single `GEMINI_API_KEY` or comma-separated `GEMINI_API_KEYS`.
- **Quota Failure Detection:** Automatically intercepts `HTTP 429 (ResourceExhausted)` errors.
- **Quarantine & 65-Second Auto-Recovery:** When a key encounters a rate limit, it is quarantined and marked `is_depleted = True` with a timestamp. Because Google AI Studio RPM rate limits reset on a 60-second sliding cycle, the pool automatically restores keys to healthy status after a **65-second cooldown** without requiring server restarts.

5.2 OpenRouter Key Pool (`OpenRouterKeyPool`)

- **Real-Time Balance Polling:** Automatically queries `https://openrouter.ai/api/v1/auth/key` to monitor credit usage and remaining balance.
- **Automatic Shift on Depletion:** Keys returning `HTTP 402 (Insufficient Credits)` or low balance are instantly rotated out in favor of healthy alternatives.

5.3 Cross-Provider Failover Matrix

High-Availability Failover Chain

If all keys in the primary **Google Gemini** pool are temporarily exhausted, requests gracefully cascade to **Groq Cloud LPUs** (Compound Mini / Qwen 3.8), and subsequently to **OpenRouter** fallback models. Users experience continuous availability with zero service interruptions.

6. GoHighLevel (GHL) REST API v2 SDK & Tools Matrix

SECTION 06

The `GHLSubAccountClient` provides a unified, production-grade SDK interacting strictly with GoHighLevel's official REST API v2:

- **Base URL:** `https://services.leadconnectorhq.com`
- **Version Header:** `Version: 2021-07-28`
- **Authentication:** `Authorization: Bearer <ACCESS_TOKEN>`

TOOL NAME	TARGET ENDPOINT	METHOD	DESCRIPTION & HYGIENE RULES
`create_contact`	`/contacts/`	POST	Creates contact with first/last name, email, E.164 phone, tags, and custom fields. Suppresses redundant `name` fields.
`search_contacts`	`/contacts/`	GET	Queries contacts by name, email, or phone. Used for pre-flight duplicate checks.
`create_pipeline`	`/opportunities/pipelines/`	POST	Builds custom sales or retention pipelines with ordered positional stages.
`get_pipelines`	`/opportunities/pipelines/`	GET	Fetches existing pipeline and stage IDs for deal card routing.
`create_opportunity`	`/opportunities/`	POST	Inserts deal card into specified pipeline and stage with monetary values and status.
`create_tag`	`/locations/{id}/tags`	POST	Adds a new tag to the location tag taxonomy.
`create_custom_field`	`/locations/{id}/customFields`	POST	Provisions fields: `TEXT`, `NUMBER`, `DATE`, or `SINGLE_OPTIONS` with defined dropdown choices.
`send_conversation_message`	`/conversations/messages`	POST	Dispatches outbound SMS or Email directly through connected sub-account telephony.
`create_contact_task`	`/contacts/{id}/tasks`	POST	Assigns actionable task with ISO due date to a contact record.
`create_contact_note`	`/contacts/{id}/notes`	POST	Appends internal staff notes to a contact record.
`setup_gym_subaccount`	Batch API Handler	POST	Deploys the full 14-field, 12-tag Gym & Fitness Center CRM taxonomy in a single command.

7. Senior GHL Solutions Architect Engineering Rules

SECTION 07

The Copilot embeds the **29 Senior Solutions Architect Rules** within its core system prompt. These rules enforce elite software engineering discipline and eliminate typical chatbot weaknesses:

1. Direct Writing Style Mandate

Zero pleasantries, zero self-introductions ("As an AI...", "I am thrilled to..."). Direct, authoritative, and actionable instructions only.

2. Strict E.164 Phone Standards

All phone numbers must strictly adhere to international E.164 standards (+1 followed by 10 digits for US/Canada) to avoid carrier deliverability drops.

3. Contact Payload Hygiene

When discrete `firstName` and `lastName` values are provided, the agent forbids sending a redundant `name` field that can cause race conditions in GHL.

4. Auth Boundary Distinction

Strictly distinguishes Sub-Account Private Integration Tokens (PIT) from Agency OAuth 2.0 applications, preventing permission scope mismatches.

5. Failure-Mode Awareness

Proactively identifies and warns users regarding duplicate contact merge rules, unverified email sending domains, and SMS compliance requirements.

6. Zero Fabrication & Query Scope

Never fabricates hypothetical endpoints. If a capability requires custom webhooks or third-party middleware (e.g. Zapier / Make), it explicitly states so.

8. Production Vertical Architecture: Gym & Fitness Centers

SECTION 08

The repository includes a turnkey vertical architecture module (`gym_architecture.py`) that provisions enterprise-grade schemas for boutique fitness studios, CrossFit gyms, and commercial fitness centers.

8.1 Custom Fields Blueprint (14 Strategic Fields)

FIELD NAME	DATA TYPE	CONFIGURED OPTIONS / RATIONALE
`Primary Fitness Goal`	SINGLE_OPTIONS	Weight Loss, Muscle Building, General Health, Athletic Performance, Post-Rehab.
`Exercise Experience`	SINGLE_OPTIONS	Beginner (0-6 mo), Intermediate (1-2 yrs), Advanced (3+ yrs).
`Exercise Limitations`	SINGLE_OPTIONS	No, Yes.
`Limitation Category`	SINGLE_OPTIONS	None, Lower Body/Knee, Upper Body/Shoulder, Back/Core, Cardio.
`Lead Score`	NUMBER	Dynamic engagement score for lead qualification (0-100).
`Lead Tier`	SINGLE_OPTIONS	Cold, Warm, Hot (evaluated via hysteresis logic).
`Membership Plan Type`	SINGLE_OPTIONS	Month-to-Month, Annual VIP, Personal Training, Student/Senior.
`Membership Status`	SINGLE_OPTIONS	Prospect, Trial Active, Member Active, Frozen, Churned.
`Total Check-ins Completed`	NUMBER	Automated counter for member retention tracking.

8.2 Dual Pipelines & Retention Hysteresis Logic

- **Gym Sales & Trial Pipeline:** `New Prospect Inquiry` ➡ `Free Pass Claimed` ➡ `Tour Booked` ➡ `Tour Completed` ➡ `Trial Active (Day 1-7)` ➡ `VIP Converted (Won)` ➡ `Recycle (Lost)`.
- **Member Retention Pipeline:** `Onboarding Week 1` ➡ `Active Consistent` ➡ `Attendance Dip (<2 Visits in 14 Days)` ➡ `At-Risk Flagged` ➡ `Save Protocol Engaged` ➡ `Churned / Frozen`.

9. Semantic Retrieval Engine (RAG) & Case Study Grounding

SECTION 09

To prevent ungrounded AI claims, `portfolio_knowledge_base.py` indexes verified agency projects from local binary documentation:

- **`KPI Scope .pdf`:** Multi-account KPI tracking, client reporting architectures, data pipelines.
- **`XortLogix_Facebook_Analytics_Dashboard_Project_Document.docx`:** Meta Ads API integration, automated reporting, webhook synchronization.

9.1 Dual-Stage Hybrid Retrieval Pipeline

1. Dense Vector Embeddings

Pre-computed dense vector representations evaluated with cosine similarity matching to identify deep conceptual relevance.

2. Sparse Term-Frequency Overlap

Fast lexical term matching that scores exact technical terminology (e.g. "Meta Marketing API", "XortLogix", "PandaCare").

🎯 **Authentic Citation Guarantee**

When a user requests case studies or proof of work, verified project excerpts are automatically injected into the system prompt. The model cites real client architectures and authentic performance metrics rather than generating synthetic claims.

10. Token Usage Tracking & Quota Monitor

SECTION 10

The `usage_tracker.py` module maintains persistent, thread-safe monitoring of all model interactions:

- **Sliding-Window Metrics:** Real-time tracking of Tokens Per Minute (TPM) and Requests Per Minute (RPM) to prevent hitting provider rate spikes.
- **Daily Quota Tracking:** Tracks daily token accumulation against standard model quotas (e.g. 1M TPM / 1,500 RPD for Gemini Flash).
- **Automated UTC Rollover:** Automatically clears and resets daily counters when the UTC calendar date transitions.
- **State Persistence:** Serializes usage telemetry to `model_usage.json` with automated backup and recovery from `model_usage.json.bak`.

11. Complete REST & SSE API Reference

SECTION 11

METHOD	ROUTE	PURPOSE	FORMAT
`GET`	`/health`	Server status & configured provider health check.	JSON
`GET`	`/api/models`	Available models catalog enriched with live usage stats.	JSON
`GET`	`/api/usage-stats`	Global token and request metrics across all models.	JSON
`GET`	`/api/openrouter/pool-status`	Real-time balance and rotation state of OpenRouter keys.	JSON
`POST`	`/api/ghl/verify-token`	Validates Sub-Account Location ID & Bearer Token.	JSON
`POST`	`/api/ghl/contacts`	Fetches contacts list from connected sub-account.	JSON
`POST`	`/api/ghl/create-contact`	Creates a contact directly without natural language prompt.	JSON
`POST`	`/api/ghl/pipelines`	Fetches pipelines and stage metadata from sub-account.	JSON
`POST`	`/api/ghl/setup-gym`	Provisions 14 custom fields, 12 tags, and 2 pipelines.	JSON
`POST`	`/api/chat-agent`	Interactive agent execution with Server-Sent Events stream.	SSE Stream

12. Deployment, Containerization & Verification

SECTION 12

12.1 Local Environment Setup

```
# 1. Clone the repository
git clone https://github.com/muhammadokashapak/Conversation-AI-Copilot.git
cd Conversation-AI-Copilot

# 2. Initialize and activate Python virtual environment
python -m venv venv
.\venv\Scripts\activate   # Windows PowerShell
source venv/bin/activate  # Linux / macOS

# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure .env file
cp .env.example .env

# 5. Launch the application
python app.py
```

12.2 Cloud Deployment (Railway, Render & Docker)

- **Railway:** Pre-configured with native `railway.json` utilizing NIXPACKS builders and automatic restart policies.
- **Heroku / Render:** Includes standard `Procfile` executing:
  `web: uvicorn app:app --host 0.0.0.0 --port ${PORT:-8080} --proxy-headers --forwarded-allow-ips="*"`
- **Docker:** Containerize via standard Python 3.10-slim image exposing port `7861` with proxy headers enabled.

12.3 System Verification & Verification Summary

✅ **System Health Verified**

The backend was verified active on `http://127.0.0.1:7861`. Endpoints `/health`, `/api/models`, and `/api/chat-agent` were validated with 200 OK status codes and successful Server-Sent Events streaming.